Smart-Camera-P160-SL — Control4 Driver

Overview

Complete Control4 driver for Slomins P160-SL IP Camera with full API integration, authentication, streaming, PTZ support, and real-time MQTT event detection.

Version: 2
Package: Slomins-indoor-P160.c4z
Minimum Control4 OS: 3.3.2+

Features

- ✅ **Full API Integration**
  - Initialize camera and retrieve public key
  - User authentication with RSA-OAEP encryption
  - Token management (temporary and exchange tokens)
  - Device listing
- ✅ **Streaming Support**
  - RTSP main stream (high quality)
  - RTSP sub stream (low quality)
  - Dynamic URL generation
- ✅ **Camera Functions**
  - Snapshot URL generation
  - PTZ controls (Pan, Tilt, Zoom)
  - Preset management
- ✅ **Security**
  - RSA-OAEP + SHA256 encryption using C4:Crypto()

  - HMAC-SHA256 signatures for API requests
  - Secure token storage

- ✅ **Real-Time Event Detection (MQTT)**
  - Motion, human, face, and stranger detection
  - Doorbell ring notifications with snapshot
  - Camera online/offline monitoring

Requirements

- **Control4 Composer Pro** (appropriate version for your platform)
- **Control4 OS** 3.3.2 or newer (for RSA encryption support)
- Network Access to API endpoint and camera
- Camera accessible on LAN

Installation

1. Install Driver Package

1. Open **Control4 Composer Pro**
2. Go to **Drivers** → **Add Driver** → **Install From File**
3. Select `Slomins-indoor-P160.c4z` and click **Install**

2. Add Device to Project

1. Go to **Project** view
2. Select a room
3. Click **Add Device**
4. Search for "Slomins P160-SL" and add it

3. Configure Properties

Set the following properties:

- **API Configuration**
  - **Base API URL**: `https://api.arpha-tech.com`
  - **Account**: Your email or phone number
- **Camera Configuration**
  - **IP Address**: Camera IP (e.g., 192.168.1.100)
  - **HTTP Port**: 80 (default)
  - **RTSP Port**: 554 (default)
  - **Username** / **Password** for camera

Quick Start Guide

Complete Setup Flow

```
1. Execute "Initialize"
   → Gets RSA public key from API
   → Stores in "Public Key" property

2. Execute "Login/Register"
   → Encrypts credentials with RSA-OAEP
   → Authenticates with API
   → Stores Auth Token

3. Execute "Get Devices"
   → Lists all devices for user
   → View results in Lua Output

4. Execute "Test Main Stream"
   → Generates RTSP URL for streaming

5. Execute "Get Snapshot URL"
   → Generates HTTP snapshot URL
```

Dynamic IP Auto-Update (Online Event Handling)

The driver includes intelligent handling of camera connectivity to ensure the correct IP address is always used.

Whenever the camera comes online, the driver automatically:

- Detects the **Camera Online** event
- Calls the **Get Devices API**
- Matches the device using **VID (Virtual ID)**
- Retrieves the latest **local IP address**
- Updates the **IP Address** property in the driver
- Pushes the updated address to the **Control4 Camera Proxy**

Real-Time Event Detection (MQTT)

The driver supports **MQTT-based real-time event streaming over SSL (port 8884)** and provides notifications for the following events:

- Motion detection with snapshot attachment
- Human detection
- Camera online/offline status monitoring

MQTT Configuration

Default Behavior

MQTT is **enabled automatically by the driver after authentication**. The driver will automatically:

1. Enable MQTT
2. Fetch MQTT credentials

3. Connect to the MQTT broker
4. Subscribe to camera event topics

You normally **do not need to enable MQTT manually**.

CldBus App Motion Detection Settings

To receive **Motion Detection** and **Human Detection** events, detection must be enabled in the **CldBus mobile app**.

Steps

1. Open the **CldBus App**
2. Select your **camera device**
3. Tap **Settings**
4. Navigate to: `Settings → Motion Detection`
5. Under **Detection Type**, select one of the following:

Detection Type	Description
All Detections	All movement events will trigger notifications. This includes general motion detection.
Human Detection	Only human movement will trigger events. Non-human motion will be ignored.

Push Notification Configuration

1. Create Push Notification

1. Open **Composer Pro**
2. Navigate to: `Agents → Push Notification`

3. Click **Add Notification** then enter a name for the notification.

Configure the following:

Setting	Value
Category	Cameras
Severity	Info / Critical
Subject	Camera Event

Click **Save**.

2. Enable Snapshot Attachment

Edit the created notification and set:

```
Attachment Type = Snapshot URL
```

3. Map Push Notifications in Programming

1. In **Composer Pro**, open **Programming**
2. Select the **Camera Driver** from the device list
3. In the **Events** section, choose the event to trigger (e.g. **Motion Detected**)
4. On the right-side panel: Click **Push Notifications**
5. From the dropdown, select the notification you created earlier
6. Drag and drop the notification into the programming area or double-click the green arrow to add it

```
Example:
WHEN Motion Detected → THEN Send Push Notification
```

Notification Flow

```
Camera Event (MQTT)
        ↓
Driver Receives Event
        ↓
Driver Processes Event
        ↓
Control4 Event Triggered
        ↓
Push Notification Agent
        ↓
Mobile Notification Sent
        ↓
Snapshot Image Attached
```

Available Actions

All actions accessible via: **Device → Actions → Select action → Execute**

Authentication & API

- **Initialize** - Get RSA public key from API
- **Login/Register** - Authenticate user and get auth token
- **Get Temp Token** - Get temporary token (300 seconds)
- **Get Exchange Token** - Exchange temp token for identity token
- **Get Devices** - List all user devices

Streaming & Snapshots

- **Test Main Stream** - Generate high quality RTSP URL
- **Test Sub Stream** - Generate low quality RTSP URL
- **Test Snapshot** - Generate HTTP snapshot URL

PTZ Controls

- **PTZ Up** / **Down** / **Left** / **Right**

API Functions

1. Initialize Camera

Command: `INITIALIZE_CAMERA`
Endpoint: `POST /api/v3/openapi/init`

Purpose: Retrieve RSA public key for encryption

What it does:

- Generates client ID and request ID
- Creates HMAC-SHA256 signature
- Requests public key from API
- Stores key in "Public Key" property

Expected Output:

```
Camera initialization succeeded
Received public key: MIGfMA...
Public key stored successfully
```

2. Login/Register

Command: `LOGIN_OR_REGISTER`
Endpoint: `POST /api/v3/openapi/auth/login-or-register`

Purpose: Authenticate user with encrypted credentials

Prerequisites:

- Must run Initialize first (needs public key)
- Account must be set in properties

What it does:

- Encrypts credentials with RSA-OAEP + SHA256
- Sends authentication request

- Stores auth token and user ID
- Token used for subsequent API calls

Encryption Process:

```
1. Create crypto object: C4:Crypto("RSA")
2. Import public key
3. Encrypt JSON: {"country_code":"US","account":"user@email.com"}
4. Generate HMAC-SHA256 signature
5. Send to API
```

Expected Output:

```
Using C4:Crypto for RSA-OAEP encryption...
Crypto object created successfully
Public key imported successfully
Encrypting data with RSA-OAEP + SHA256...
Encryption successful!
Login/Register succeeded
Auth token stored
User ID: 12345
```

3. Get Temp Token

Command: `GET_TEMP_TOKEN`

Endpoint: `POST /api/v3/openapi/temperate-token`

Purpose: Get temporary token (300 seconds duration)

Request:

```
{
  "duration": 300
}
```

Response (example):

```
{
  "code": 20000,
  "message": "success",
```

```
  "data": {
    "token": "Em2SB9ijEYrQimb0v8irW/WTOZm67dHHeqArhGBom9M="
  }
}
```

Result: Token stored in "Temp Token" property

4. Get Exchange Token

Command: `GET_EXCHANGE_TOKEN`
Endpoint: `POST /api/v3/openapi/auth/exchange-identity`

Purpose: Exchange temporary token for identity token

Request (example):

```
{
  "token": "Em2SB9ijEYrQimb0v8irW/WTOZm67dHHeqArhGBom9M="
}
```

5. Get Devices

Command: `GET_DEVICES`
Endpoint: `GET /api/v3/openapi/devices-v2`

Purpose: List all devices for authenticated user

Headers:

```
Authorization: Bearer <auth_token>
```

Response (example):

```
{
  "code": 20000,
  "message": "success",
  "data": {
    "devices": [
      {
        "device_id": "...",
```

```
        "device_name": "...",
        "device_type": "camera",
        "status": "online"
      }
    ]
  }
}
```

Result: Full device list printed in Lua Output

Streaming & Snapshot URLs

Main Stream: `rtsp://<ip>:554/streamtype=1`

Sub Stream: `rtsp://<ip>:554/streamtype=0`

Snapshot URL: `http://[user:pass@]<ip>:<port>/snap.jpg`

Properties

Input Properties (Configure These)

Property	Type	Default	Description
Base API URL	STRING	https://api.arpha-tech.com	API endpoint
Account	STRING	-	User email or phone
IP Address	STRING	192.168.1.100	Camera IP
HTTP Port	INTEGER	80	HTTP port
RTSP Port	INTEGER	554	RTSP port
Username	STRING	-	Camera username
Password	STRING	-	Camera password

Property	Type	Default	Description
Snapshot Path	STRING	/snap.jpg	Snapshot path
Enable MQTT	LIST	True	Enable/disable MQTT event streaming

Output Properties (Read-Only/Managed)

- Status — Current operation status
- Public Key — RSA public key from API
- Client ID — Generated client ID
- Auth Token — Authentication token
- Temp Token — Temporary token
- Exchange Token — Identity token
- Main Stream URL — High quality RTSP URL
- Sub Stream URL — Low quality RTSP URL
- Online — Camera online status

Workflow Examples

Workflow 1: Initial Setup & Authentication

```
1. Set "Base API URL" property
2. Set "Account" property (your email)
3. Execute "Initialize"
   → Gets public key
4. Execute "Login/Register"
   → Authenticates user
   → Stores Auth Token
   → MQTT auto-connects after auth
5. Execute "Get Devices"
   → Lists your devices
```

Workflow 2: Token Management

```
1. Execute "Get Temp Token"
   → Gets 300-second token
2. Execute "Get Exchange Token"
   → Exchanges for identity token
3. Use tokens for custom API calls
```

Workflow 3: Streaming Setup

```
1. Set "IP Address" property
2. Execute "Test Main Stream"
   → rtsp://192.168.1.100:554/streamtype=1
3. Execute "Test Sub Stream"
   → rtsp://192.168.1.100:554/streamtype=0
4. Use URLs in Control4 video configuration or external VMS
```

Workflow 4: Snapshot Testing

```
1. Set "IP Address", "Username", "Password"
2. Execute "Get Snapshot URL"
3. URL sent to camera proxy
4. Test URL in browser to verify
```

RSA Encryption

How it works: The driver uses **Control4's C4:Crypto() API** for RSA-OAEP + SHA256 encryption.

```
-- Create RSA crypto object
local crypto = C4:Crypto("RSA")

-- Import public key (from Initialize)
crypto:ImportPublicKey(publicKey)

-- Encrypt credentials
local encrypted = crypto:Encrypt(data, {
    scheme = "oaep",
```

```
    hash = "sha256"
})
```

Requirements:

- Control4 OS 3.x or newer
- Public key from Initialize API
- PEM formatted public key

Fallback Mode: If C4:Crypto() is unavailable, set "Pre-Encrypted Post Data" property and use that value for testing.

Logging & Troubleshooting

View Logs

1. Open **Composer Pro**
2. Go to **Lua Output** window
3. Execute any action and watch detailed logs

Common Issues

- **No public key available** — Run Initialize first; check Base API URL and network
- **Failed to create C4:Crypto object** — Control4 OS too old (need 3.x+)
- **Login failed** — Verify Account property and public key
- **IP Address not set** — Configure property and verify camera reachability
- **No auth token available** — Run Login/Register
- **MQTT not receiving events** — Verify Enable MQTT = True and port 8884 access

---

Building the Driver

Build package:

```
.
# Build package (PowerShell):
.\build-c4z.ps1
```

Manual build:

```
# Zip driver files
$files = @(
    "driver.lua",
    "driver.xml",
    "mqtt_manager.lua",
    "README.md",
    "CldBusApi\*.lua"
)

Compress-Archive -Path $files -DestinationPath "temp.zip"
Rename-Item "temp.zip" "Smart-Camera-P160-SL.c4z"
```

Development Notes

- **Helper Libraries (CldBusApi/)**
  - auth.lua — Authentication utilities
  - dkjson.lua — JSON encoding/decoding
  - http.lua — HTTP request helpers
  - sha256.lua — HMAC-SHA256 implementation
  - transport_c4.lua — Control4 transport adapter
  - util.lua — UUID generation, HMAC functions
- **Signature Generation** — HMAC-SHA256 used for API request signatures
- **UUID Generation** — util.uuid_v4()

Version History

v3(Current)

- Implemented functionality to store all incoming events in the database.

v2.0

- Fixed version display showing 0 in Control4 Manage Drivers interface
- Fixed device specific conditionals labels not showing in Control4 Composer

v0.1.4

- Removed hardcoded AppName

v1.0.3

- Updated Control4 Camera Programming Options
- **Device Events**: Added "Memory Card not detected"
- **Commands Added**: Take Snapshot, Initialize Camera *, Unmute Mic, Mute Mic, Turn up speaker volume, Turn down speaker volume
- **Commands Removed**: AWAKE_CAMERA, TEST_SNAPSHOT, TEST_MAIN_STREAM, TEST_SUB_STREAM, BIND_DEVICE, LOGIN_OR_REGISTER, GET_TEMP_TOKEN, GET_EXCHANGE_TOKEN, GET_DEVICES, DISCOVER_CAMERAS, DISCOVER_CAMERAS_SDDP, SET_DEVICE_PROPERTY, SEND_NOTIFICATION
- **Conditions Added**: If sensing motion, If not sensing motion, If Mic is muted, If Mic is unmuted, If Speaker volume is 1-10

v0.1.0

- Complete API integration
- RSA-OAEP encryption with C4:Crypto()
- User authentication
- Token management
- Device listing
- RTSP streaming URLs, snapshot support, PTZ controls
- MQTT real-time event detection (SSL, port 8884)

Support & Contact

- **Driver Version:** 2
- **Maintainer:** Slomins
- **Website:** www.slomins.com
- **Email:** support@slomins.com

License

End of Documentation